

BANCO DE DADOS

com Microsoft SQL Server

AULA 06

Consultas Avançadas

LIKE · BETWEEN · IN · IS NULL · DISTINCT · Alias

LIKE

Wildcards

BETWEEN

IN / NOT IN

IS NULL

DISTINCT

Alias avançado

COALESCE



Objetivos da Aula 06



Buscar padrões de texto com LIKE e wildcards (% e _)



Eliminar duplicatas eficientemente com DISTINCT



Filtrar intervalos com BETWEEN (números, datas e texto)



Criar aliases descritivos para colunas e expressões



Verificar pertinência em listas com IN e NOT IN



Substituir NULL por valores padrão com COALESCE e ISNULL



Identificar e tratar valores ausentes com IS NULL / IS NOT NULL



Combinar todos os operadores em consultas reais no SISTEMA_VENDAS



Revisão — Aula 05: SELECT e WHERE

? Qual é a ordem de execução do SELECT?

✓ FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → OFFSET/FETCH

? Qual a precedência entre AND e OR?

✓ AND tem precedência sobre OR — sempre use parênteses para deixar explícito

? O que faz DISTINCT?

✓ Elimina linhas duplicadas no resultado do SELECT (não afeta a tabela)

? Como funciona OFFSET/FETCH?

✓ OFFSET pula N linhas, FETCH NEXT M pega as próximas M — paginação ANSI

? Por que evitar SELECT * em produção?

✓ Retorna colunas desnecessárias, mais lento, quebra se a tabela mudar

LIKE — Buscas por Padrão de Texto

LIKE compara uma coluna de texto a um padrão usando wildcards (curingas). É o operador ideal para buscas parciais — quando você sabe parte do valor mas não o valor exato.

Wildcards disponíveis:



Qualquer sequência

'A%' → Começa com A

'%Silva' → Termina com Silva

'%note%' → Contém 'note'



Exatamente 1 caractere

'_P' → 2 chars, termina em P

'A_a' → 3 chars: A, qualquer, a

'__@%' → 2 chars antes do @



Lista de caracteres

'[AE]na' → Ana ou Ena

'[0-9]%' → Começa com dígito

'[^A-Z]%' → Não começa com maiúsc



LIKE — Exemplos no SISTEMA_VENDAS

```
USE SISTEMA_VENDAS;

-- Clientes cujo nome começa com 'A' ou 'B'
SELECT nome, email FROM CLIENTES
WHERE nome LIKE 'A%' OR nome LIKE 'B%';

-- Clientes com 'Silva' em qualquer posição do nome
SELECT nome, cidade FROM CLIENTES
WHERE nome LIKE '%Silva%';

-- Emails do domínio @email.com
SELECT nome, email FROM CLIENTES
WHERE email LIKE '%@email.com';

-- Produtos com exatamente 2 palavras no nome (tem espaço)
SELECT nome_produto FROM PRODUTOS
WHERE nome_produto LIKE '% %'; -- tem pelo menos um espaço

-- NOT LIKE: exclui o padrão
SELECT nome, email FROM CLIENTES
WHERE email NOT LIKE '%@hotmail%'
AND email IS NOT NULL;

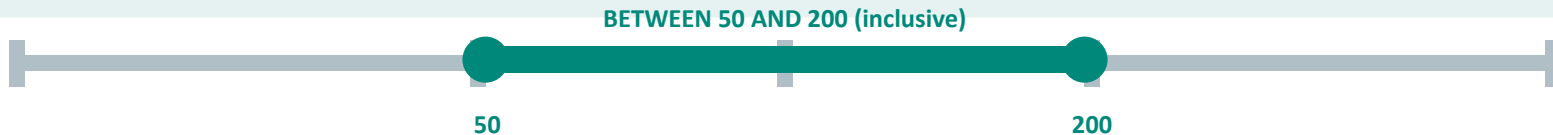
-- Busca de produto ignorando maiúsculas (UPPER forçado)
SELECT nome_produto, preco FROM PRODUTOS
WHERE UPPER(nome_produto) LIKE '%NOTE%'; -- encontra Notebook

-- Escape de caractere especial: buscar '%' literal
WHERE descricao LIKE '%10[%] desconto%' ESCAPE '[';

-- ou: WHERE descricao LIKE '%10!% desconto%' ESCAPE '!';
```

BETWEEN — Intervalos Inclusivos

BETWEEN valor_min AND valor_max — retorna registros onde o valor está DENTRO do intervalo, incluindo os extremos. Equivale a: coluna >= min AND coluna <= max.



Com Números

```
SELECT nome_produto, preco
FROM   PRODUTOS
WHERE  preco
      BETWEEN 50.00 AND 500.00;

-- >= 50 AND <= 500
```

Com Datas

```
SELECT id_pedido, data_pedido
FROM   PEDIDOS
WHERE  data_pedido
      BETWEEN '2025-01-01'
          AND '2025-12-31';
```

NOT BETWEEN

```
SELECT nome_produto, preco
FROM   PRODUTOS
WHERE  preco
      NOT BETWEEN 100 AND 1000;

-- < 100 OR > 1000
```



BETWEEN com datas: use datas completas! '2025-12-31' exclui o horário 23:59. Use < '2026-01-01' para incluir o dia todo, ou CONVERT(DATE, coluna) = '2025-12-31'.

IN e NOT IN — Pertinência a Listas

IN verifica se um valor PERTENCE a uma lista — é mais legível que múltiplos OR. NOT IN verifica se o valor NÃO está na lista. Ambos aceitam listas literais ou subqueries.

✗ Sem IN — verboso e propenso a erro

```
SELECT nome FROM CLIENTES
WHERE estado = 'SP'
OR estado = 'RJ'
OR estado = 'MG'
OR estado = 'PR';
```

✓ Com IN — legível e manutenível

```
SELECT nome FROM CLIENTES
WHERE estado IN (
    'SP', 'RJ', 'MG', 'PR'
);
```

IN com subquery (dinâmico)

```
SELECT nome FROM CLIENTES
WHERE id_cliente IN (
    SELECT id_cliente
    FROM PEDIDOS);
```

⚠ NOT IN + NULL = armadilha!

```
SELECT nome FROM CLIENTES
WHERE id_cliente NOT IN (
    SELECT id_cliente
    FROM PEDIDOS -- sem filtro!
    -- NULL causa retorno vazio!
```

IS NULL e IS NOT NULL — Tratando Valores Ausentes

NULL não é zero, não é string vazia, não é espaço — é a AUSÊNCIA de valor. Por isso, `NULL = NULL` retorna FALSE no SQL. A única forma de testar NULL é com `IS NULL` ou `IS NOT NULL`.

Como o SQL avalia NULL:

`NULL = NULL`

FALSE ❌

`NULL <> NULL`

FALSE ❌

`NULL = 0`

FALSE ❌

`NULL = ''`

FALSE ❌

`NULL IS NULL`

TRUE ✅

`1 IS NOT NULL`

TRUE ✅

Exemplos práticos:

```
SELECT nome FROM CLIENTES WHERE email IS NULL;      -- sem e-mail
SELECT nome FROM CLIENTES WHERE email IS NOT NULL;  -- com e-mail
SELECT nome, ISNULL(email, 'Não informado') AS contato FROM CLIENTES;
```



DISTINCT — Valores Únicos e Combinações

```
-- DISTINCT: elimina linhas com valores duplicados no resultado

-- Quais estados têm clientes cadastrados?
SELECT DISTINCT estado FROM CLIENTES ORDER BY estado;

-- Quais cargos existem na empresa?
SELECT DISTINCT cargo FROM FUNCIONARIOS ORDER BY cargo;

-- DISTINCT em múltiplas colunas (combinação única)
SELECT DISTINCT estado, cidade FROM CLIENTES
ORDER BY estado, cidade;

-- Retorna cada par (estado, cidade) único

-- COUNT com DISTINCT: quantos valores únicos?
SELECT COUNT(DISTINCT estado)          AS estados_distintos,
       COUNT(DISTINCT id_categoria) AS categorias_usadas
FROM   CLIENTES, PRODUTOS;

-- DISTINCT vs GROUP BY: qual é mais eficiente?
SELECT DISTINCT estado FROM CLIENTES; -- simples, direto
SELECT estado      FROM CLIENTES GROUP BY estado; -- mesmo resultado

-- GROUP BY é mais flexível pois permite funções de agregação

-- Quando DISTINCT não é a melhor opção:
-- Se precisa de contagem por grupo → use GROUP BY (Aula 09)
-- Se a tabela é grande → DISTINCT pode ser lento (ordena tudo)
-- Se os dados são únicos por design → DISTINCT é desnecessário
```

Alias — Renomeando com Propósito

Alias (apelidos) tornam as queries mais legíveis, os relatórios mais profissionais e o código mais manutenível. São essenciais em JOINS (Aula 10) e em expressões calculadas.

Alias de COLUNA simples

```
SELECT nome AS 'Nome do Cliente',  
       estado AS UF  
FROM   CLIENTES;
```

💡 AS pode ser omitido: nome 'Nome do Cliente'. Use [] para nomes com espaços especiais.

Alias de EXPRESSÃO calculada

```
SELECT preco * estoque AS valor_em_estoque,  
       preco * 0.90 AS preco_com_desconto  
FROM   PRODUTOS;
```

💡 Alias de expressão não pode ser usado no WHERE (ORDER BY sim!).

Alias de TABELA

```
SELECT c.nome, c.estado  
FROM   CLIENTES AS c  
WHERE  c.estado = 'SP';
```

💡 Alias de tabela é OBRIGATÓRIO quando a mesma tabela aparece mais de uma vez (self join).

Alias em SUBQUERY

```
SELECT sub.estado, sub.qtd  
FROM   (SELECT estado, COUNT(*) AS qtd  
        FROM CLIENTES GROUP BY estado) AS sub
```

💡 `ORDER BY sub.qtd DESC;`
Subqueries no FROM DEVEM ter alias. Esta técnica é chamada de 'derived table' (tabela derivada).

COALESCE e ISNULL — Tratando NULL nos Resultados

```
-- ISNULL(expr, substituto): simples e direto
SELECT nome,
       ISNULL(email,      'Não informado') AS email,
       ISNULL(telefone, 'Sem telefone')   AS telefone
FROM   CLIENTES;

-- COALESCE(e1,e2,...): retorna o 1º não-NULL (padrão ANSI)
-- Tenta celular, depois telefone, depois 'Sem contato'
SELECT nome,
       COALESCE(email, 'Sem e-mail') AS contato
FROM   CLIENTES;

-- Uso em ORDER BY: colocar NULLs no final
SELECT nome, email FROM CLIENTES
ORDER BY COALESCE(email, 'ZZZ'); -- NULL vira 'ZZZ' = vai pro fim

-- Contar NULLs vs não-NULLs
SELECT COUNT(*)          AS total,
       COUNT(email)      AS com_email,
       COUNT(*) - COUNT(email) AS sem_email
FROM   CLIENTES;

-- Filtro: incluir NULLs OU valor específico
SELECT nome, id_funcionario FROM PEDIDOS
WHERE id_funcionario IS NULL OR id_funcionario = 1;

-- Sem funcionário ou pedidos do João
```

Combinando Operadores — Consultas Reais de Negócio

```
-- CASO 1: Busca de produtos para campanha de verão
-- (Esportes ou Roupas, preço até R$300, com estoque)
SELECT nome_produto AS produto,
       preco        AS [Preço],
       estoque      AS [Em estoque],
       ISNULL(descricao,'Sem descrição') AS descricao
FROM   PRODUTOS
WHERE  id_categoria IN (2, 5)
AND    preco BETWEEN 50.00 AND 300.00
AND    estoque > 0
ORDER BY preco ASC;

-- CASO 2: Clientes para campanha de e-mail (SP e RJ com email)
SELECT nome                AS [Nome],
       email               AS [E-mail],
       CONCAT(cidade,'/',estado) AS [Localização],
       DATEDIFF(DAY,data_cadastro,GETDATE()) AS [Dias cadastrado]
FROM   CLIENTES
WHERE  estado IN ('SP','RJ')
AND    email IS NOT NULL
AND    nome LIKE '%a%' -- tem letra 'a' no nome
ORDER BY [Dias cadastrado] DESC;

-- ⚠ alias entre [] não funciona no ORDER BY aqui
-- Correto: ORDER BY DATEDIFF(DAY,data_cadastro,GETDATE()) DESC;
```



Exercícios Propostos — Aula 06

01

Básico

Use LIKE para encontrar: (a) clientes cujo nome contém 'a' (minúscula ou maiúscula), (b) produtos que começam com 'Note' ou 'Fone', (c) emails que terminam em '.com'.

02

Básico

Liste todos os produtos com preço BETWEEN R\$100 e R\$1.000. Depois, use NOT BETWEEN para mostrar os que estão fora dessa faixa. Compare os dois resultados.

03

Médio

Mostre todos os clientes que: pertencem aos estados IN ('SP','RJ','MG') E têm email IS NOT NULL. Use ISNULL para substituir o telefone NULL por 'Não informado'.

04

Médio

Crie uma query de auditoria: mostre COUNT(*) total de clientes, COUNT(email) com email, e a diferença (sem email). Depois, use DISTINCT para listar os estados únicos com clientes.

05

Avançado

Monte uma busca avançada de produtos: nome LIKE '%o%', preço BETWEEN 50 AND 500, id_categoria NOT IN (3,4). Use alias descritivos, ISNULL na descrição e ordene por preço. Depois, reescreva usando uma subquery no FROM com alias.



Respostas Comentadas — Exercícios 03 e 05

```
-- EX 03: Clientes SP/RJ/MG com email, telefone tratado
SELECT nome                AS [Nome],
       email               AS [E-mail],
       estado              AS UF,
       ISNULL(telefone,'Não informado') AS [Telefone]
FROM   CLIENTES
WHERE  estado IN ('SP','RJ','MG')
AND    email IS NOT NULL
ORDER BY estado, nome;
```

-- EX 05: Busca avançada com subquery no FROM

-- Versão direta:

```
SELECT nome_produto        AS Produto,
       preco               AS [Preço (R$)],
       ISNULL(descricao,'Sem descrição') AS Descricao
FROM   PRODUTOS
WHERE  nome_produto LIKE '%o%'
AND    preco BETWEEN 50 AND 500
AND    id_categoria NOT IN (3, 4)
ORDER BY preco;
```

-- Com subquery no FROM (derived table — prévia de View):

```
SELECT sub.Produto, sub.[Preço (R$)], sub.Descricao
FROM (
    SELECT nome_produto AS Produto, preco AS [Preço (R$)],
           ISNULL(descricao,'Sem descrição') AS Descricao
    FROM   PRODUTOS
    WHERE  nome_produto LIKE '%o%'
           AND    preco BETWEEN 50 AND 500
           AND    id_categoria NOT IN (3, 4)
) AS sub
ORDER BY sub.[Preço (R$)];
```



Desafio da Aula 06

Contexto: O setor de marketing precisa de listas segmentadas para campanhas. Cada lista tem critérios específicos combinando LIKE, BETWEEN, IN e IS NULL.

Segmentações solicitadas:

LIKE + IS NOT NULL

Clientes com sobrenome (parte após espaço) contendo 'a' ou 'o' E que têm email cadastrado. Exibir nome, email e cidade.

IN + BETWEEN

Produtos das categorias Eletrônicos e Esportes (IN) com preço entre R\$150 e R\$1.000 (BETWEEN). Exibir nome, preço formatado e estoque.

IS NULL + NOT IN

Clientes sem email cadastrado (IS NULL) que NÃO moram em SP nem RJ (NOT IN). Listar nome, cidade, estado com telefone tratado com ISNULL.

DISTINCT

Lista de cidades únicas (DISTINCT) onde há clientes com nome começando por vogal (A, E, I, O, U). Use LIKE '[AEIOU]%'

BÔNUS

Combine TUDO: produtos cujo nome LIKE '%Pro%' OU estejam na categoria IN(1,5), com preço NOT BETWEEN 3.000 AND 10.000, exibindo ISNULL(descricao,'—') como descrição.



Resumo — Aula 06: Consultas Avançadas



LIKE

% = qualquer texto; _ = um caractere; [] = lista. Case-insensitive no SQL Server



BETWEEN

Intervalo inclusivo. Com datas: atenção ao horário — use < próximo dia para cobrir tudo



IN / NOT IN

Pertinência a lista. NOT IN com NULL na subquery = zero linhas! Use IS NOT NULL



IS NULL

Única forma correta de testar NULL. = NULL sempre retorna FALSE em SQL Server



DISTINCT

Linhas únicas no resultado. Em múltiplas colunas: combinação única. COUNT(DISTINCT col)



Alias

AS para colunas e tabelas. Subquery no FROM deve ter alias. Alias não serve no WHERE



ISNULL / COALESCE

ISNULL(col,sub): simples. COALESCE(e1,e2,...): padrão ANSI, múltiplos valores



Armadilhas

NOT IN + NULL, BETWEEN + datas, alias no WHERE, DISTINCT custoso em tabelas grandes



Próxima Aula 07: Funções SQL

Aula 07 — O que vem por aí:



COUNT, SUM, AVG, MIN, MAX

Agregação de dados para relatórios e análises



Funções de Texto

UPPER, LOWER, CONCAT, SUBSTRING, REPLACE e mais



Funções de Data

GETDATE, DATEADD, DATEDIFF, YEAR, MONTH, FORMAT



CONVERT e FORMAT

Formatar datas e números para exibição



ISNULL, COALESCE, NULLIF

Tratamento avançado de NULL em funções



Funções combinadas

Aninhar funções para transformações complexas



Para a próxima aula: conclua o desafio de segmentação e pense em que outras consultas de busca você faria no SISTEMA_VENDAS.